

Queueing Systems: Analysis and Simulation

Karthik Varam

April 2026

1 Introduction

Queueing theory studies stochastic systems involving waiting lines and plays an important role in computer systems, telecommunications, and service operations.

Queueing models are commonly described using Kendall notation $A/B/c$, where A denotes the arrival process, B the service time distribution, and c the number of servers. In particular, M denotes exponential (Markovian) interarrival or service times, while G denotes a general distribution.

In this project, we develop the mathematical framework of queueing systems using continuous-time Markov chains (CTMCs), derive key analytical results, and validate them through simulation.

1.1 The Balance Method

The balance method is used to determine the stationary distribution of a stochastic process by equating the flow of probability into each state with the flow out of that state.

In steady state, the probabilities $\{p_j\}$ satisfy

$$(\text{rate of flow into state } j) = (\text{rate of flow out of state } j).$$

For birth–death processes, this leads to the balance equations

$$\lambda_j p_j = \mu_{j+1} p_{j+1},$$

which provide a recursive relationship between successive probabilities.

Solving these equations together with the normalization condition

$$\sum_{j=0}^{\infty} p_j = 1$$

yields the stationary distribution.

2 M/M/1 Queue: Analysis

2.1 Model Description

We consider an $M/M/1$ queueing system.

- Customers arrive according to a Poisson process with rate λ
- Service times are exponentially distributed with rate μ
- There is a single server operating on a first-come, first-served basis

Let $\{X_t\}_{t \geq 0}$ denote the number of customers in the system at time t . The process $\{X_t\}$ takes values in $\mathbb{N}_0 = \{0, 1, 2, \dots\}$.

2.2 Stationary Distribution

Let

$$p_j = \lim_{t \rightarrow \infty} \mathbb{P}(X_t = j)$$

denote the long-run probability of being in state j , assuming the limit exists.

Balance Equations

In steady state, the flow into each state must equal the flow out of that state.

For $j \geq 1$, transitions into state j occur from:

- state $j - 1$ via arrivals at rate λ
- state $j + 1$ via service completions at rate μ

Transitions out of state j occur at total rate $\lambda + \mu$.

Thus, the balance equations are

$$\lambda p_{j-1} + \mu p_{j+1} = (\lambda + \mu) p_j, \quad j \geq 1.$$

For $j = 0$, we have

$$\mu p_1 = \lambda p_0.$$

Solution

A simpler set of equations is given by the detailed balance condition:

$$\lambda p_{j-1} = \mu p_j, \quad j \geq 1.$$

This gives the recursion

$$p_j = \frac{\lambda}{\mu} p_{j-1}.$$

Iterating,

$$p_j = \left(\frac{\lambda}{\mu}\right)^j p_0.$$

Normalization

Using $\sum_{j=0}^{\infty} p_j = 1$,

$$\sum_{j=0}^{\infty} \left(\frac{\lambda}{\mu}\right)^j p_0 = 1.$$

This is a geometric series, which converges if

$$\rho = \frac{\lambda}{\mu} < 1.$$

Thus,

$$p_0 = 1 - \rho,$$

and hence

$$p_j = (1 - \rho)\rho^j, \quad j \geq 0.$$

2.3 Expected Number of Customers

The expected number of customers in steady state is

$$\mathbb{E}[N] = \sum_{j=0}^{\infty} j p_j.$$

Substituting $p_j = (1 - \rho)\rho^j$,

$$\mathbb{E}[N] = (1 - \rho) \sum_{j=0}^{\infty} j \rho^j.$$

Using

$$\sum_{j=0}^{\infty} j x^j = \frac{x}{(1 - x)^2}, \quad |x| < 1,$$

we obtain

$$\mathbb{E}[N] = (1 - \rho) \frac{\rho}{(1 - \rho)^2}.$$

Thus,

$$\mathbb{E}[N] = \frac{\rho}{1 - \rho}.$$

Substituting $\rho = \frac{\lambda}{\mu}$,

$$\mathbb{E}[N] = \frac{\lambda}{\mu - \lambda}, \quad \lambda < \mu.$$

2.4 Interpretation

The condition $\rho = \frac{\lambda}{\mu} < 1$ ensures stability of the system. As λ approaches μ , the expected number of customers grows rapidly, indicating increasing congestion in the system.

2.5 Simulation Methodology

We simulate the CTMC and estimate:

- Limiting probabilities p_j using time proportions
- $\mathbb{E}[N]$ using time averages

2.6 R Implementation

We simulate the $M/M/1$ queue as a continuous-time Markov process and estimate key quantities using time averages.

```
# Simulation
simulate_mm1 <- function(lambda, mu, max_events = 100000){
  t <- 0; n <- 0
  times <- numeric(max_events)
  states <- numeric(max_events)

  for(i in 1:max_events){
    rate <- if(n == 0) lambda else (lambda + mu)
    dt <- rexp(1, rate)
    t <- t + dt

    times[i] <- t
    states[i] <- n

    if(n == 0){
      n <- 1
    } else {
      if(runif(1) < lambda/(lambda+mu)) n <- n+1 else n <- n
        -1
    }
  }
  list(times=times, states=states)
}

# Estimation
estimate_pj <- function(times, states, max_state=10){
  durations <- diff(c(0, times))
  total_time <- sum(durations)
  sapply(0:max_state, function(j)
    sum(durations[states==j]) / total_time)
}

estimate_EN <- function(times, states){
  durations <- diff(c(0, times))
  sum(states * durations) / sum(durations)
}
```

2.7 Results

```
lambda <- 2; mu <- 3

sim <- simulate_mm1(lambda, mu)

p_hat <- estimate_pj(sim$times, sim$states)
EN_sim <- estimate_EN(sim$times, sim$states)
```

The simulated estimates are compared with the theoretical values

$$p_j = (1 - \rho)\rho^j, \quad \mathbb{E}[N] = \frac{\lambda}{\mu - \lambda}.$$

The estimated and theoretical values of the expected number of customers are:

$$\text{Simulated } \mathbb{E}[N] \approx 1.9627, \quad \text{Theoretical } \mathbb{E}[N] = 2.$$

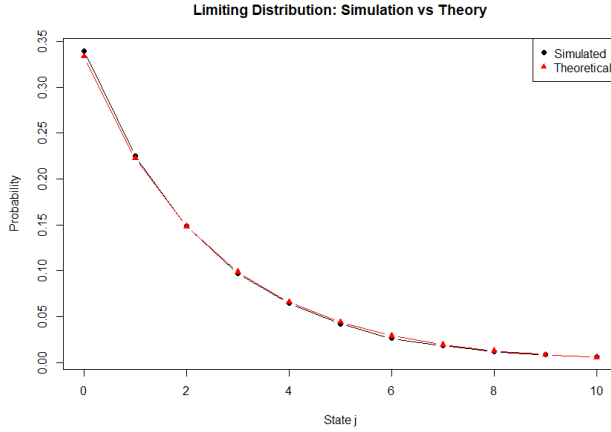
The results show close agreement between simulation and theory. The results show close agreement between simulation and theory.

2.8 Visualization

```
# Plot comparison
plot(0:max_state, p_hat, type="b", pch=16,
     xlab="State j", ylab="Probability",
     main="M/M/1: Simulation vs Theory")

lines(0:max_state, p_true, col="red", type="b", pch=17)

legend("topright",
      legend=c("Simulated", "Theoretical"),
      col=c("black", "red"),
      pch=c(16,17))
```



2.9 Conclusion

The simulation results closely match the theoretical expressions

$$p_j = (1 - \rho)\rho^j, \quad \mathbb{E}[N] = \frac{\lambda}{\mu - \lambda}.$$

The empirical distribution of states obtained from the simulation aligns well with the geometric stationary distribution, confirming that the system converges to its steady-state behaviour over time.

Furthermore, the simulated estimate of $\mathbb{E}[N]$ is in strong agreement with the theoretical value, validating the time-average approach used in the simulation.

These results support the assumption that

$$p_j = \lim_{t \rightarrow \infty} \mathbb{P}(X_t = j)$$

exists and is independent of the initial state. This demonstrates the ergodic nature of the $M/M/1$ queue under the stability condition $\rho < 1$.

Finally, we observe that as λ approaches μ , the expected number of customers increases significantly, highlighting the impact of system load on congestion.

3 M/M/c Queue: Analysis

3.1 Model Description

We now consider an $M/M/c$ queueing system with c identical servers.

- Arrivals follow a Poisson process with rate λ
- Service times are exponentially distributed with rate μ

- There are c parallel servers

Let $\{X_t\}_{t \geq 0}$ denote the number of customers in the system.

3.2 Transition Structure

When the system is in state i :

- Arrivals occur at rate λ , so

$$P_{i,i+1}(h) = \lambda h + o(h)$$

- Departures occur at rate $\min(i, c)\mu$, since at most c customers can be served simultaneously.

When the system is in state i , there are $\min(i, c)$ customers in service. Each customer in service has an exponentially distributed service time with rate μ , and these service times are independent.

Let $S_1, S_2, \dots, S_{\min(i, c)}$ denote the remaining service times. The time until the next departure is given by

$$\min\{S_1, S_2, \dots, S_{\min(i, c)}\}.$$

Since the minimum of independent exponential random variables with rate μ is again exponential with rate equal to the sum of the rates, we have

$$\min\{S_1, \dots, S_{\min(i, c)}\} \sim \text{Exp}(\min(i, c)\mu).$$

Therefore, the total departure rate from state i is $\min(i, c)\mu$, and hence

$$P_{i,i-1}(h) = \min(i, c)\mu h + o(h).$$

Hence, the process is a birth–death process with

$$\lambda_i = \lambda, \quad \mu_i = \min(i, c)\mu.$$

3.3 Stationary Distribution

Let

$$p_j = \lim_{t \rightarrow \infty} \mathbb{P}(X_t = j)$$

denote the stationary distribution.

Balance Equations

Using detailed balance,

$$\lambda p_{j-1} = \mu_j p_j,$$

where

$$\mu_j = \min(j, c)\mu.$$

Case 1: $0 \leq j < c$

Here, $\mu_j = j\mu$, so

$$\lambda p_{j-1} = j\mu p_j.$$

Thus,

$$p_j = \frac{\lambda}{j\mu} p_{j-1}.$$

Iterating,

$$p_j = \frac{1}{j!} \left(\frac{\lambda}{\mu} \right)^j p_0.$$

Case 2: $j \geq c$

Here, $\mu_j = c\mu$, so

$$\lambda p_{j-1} = c\mu p_j.$$

Thus,

$$p_j = \frac{\lambda}{c\mu} p_{j-1}.$$

Iterating from $j = c$,

$$p_j = \frac{1}{c!} \left(\frac{\lambda}{\mu} \right)^c \left(\frac{\lambda}{c\mu} \right)^{j-c} p_0.$$

Normalization

Using $\sum_{j=0}^{\infty} p_j = 1$, we obtain

$$p_0 \left[\sum_{j=0}^{c-1} \frac{1}{j!} \left(\frac{\lambda}{\mu} \right)^j + \frac{1}{c!} \left(\frac{\lambda}{\mu} \right)^c \sum_{j=c}^{\infty} \left(\frac{\lambda}{c\mu} \right)^{j-c} \right] = 1.$$

The second sum is geometric, giving

$$\sum_{j=c}^{\infty} \left(\frac{\lambda}{c\mu} \right)^{j-c} = \frac{1}{1 - \frac{\lambda}{c\mu}}.$$

Thus,

$$p_0 = \left[\sum_{j=0}^{c-1} \frac{1}{j!} \left(\frac{\lambda}{\mu} \right)^j + \frac{1}{c!} \left(\frac{\lambda}{\mu} \right)^c \frac{1}{1 - \frac{\lambda}{c\mu}} \right]^{-1}.$$

3.4 Final Expression

The stationary distribution is given by

$$p_j = \begin{cases} \frac{1}{j!} \left(\frac{\lambda}{\mu}\right)^j p_0, & 0 \leq j < c, \\ \frac{1}{c!} \left(\frac{\lambda}{\mu}\right)^c \left(\frac{\lambda}{c\mu}\right)^{j-c} p_0, & j \geq c. \end{cases}$$

3.5 Stability Condition

For the series to converge, we require

$$\frac{\lambda}{c\mu} < 1.$$

This ensures that the system is stable.

3.6 Expected Number of Customers

Let $\rho = \frac{\lambda}{c\mu}$ denote the traffic intensity, and assume $\rho < 1$ for stability.

The expected number of customers in the system can be decomposed as

$$\mathbb{E}[N] = L = L_q + L_s,$$

where:

- L_q is the expected number of customers in the queue
- L_s is the expected number of customers in service

3.7 Expected Number of Customers: Derivation

Let $\rho = \frac{\lambda}{c\mu} < 1$.

Customers in Service

In steady state, the long-run departure rate must equal the arrival rate λ . Each customer in service is served at rate μ , so if L_s denotes the expected number of customers in service, then

$$\mu L_s = \lambda.$$

Thus,

$$L_s = \frac{\lambda}{\mu}.$$

Probability of Waiting

An arriving customer has to wait if and only if all c servers are busy, i.e., the system is in a state $j \geq c$.

Thus,

$$P_{\text{wait}} = \sum_{j=c}^{\infty} p_j.$$

Using the stationary distribution derived earlier, for $j \geq c$:

$$p_j = \frac{1}{c!} \left(\frac{\lambda}{\mu} \right)^c \left(\frac{\lambda}{c\mu} \right)^{j-c} p_0.$$

Hence,

$$P_{\text{wait}} = p_0 \cdot \frac{1}{c!} \left(\frac{\lambda}{\mu} \right)^c \sum_{j=c}^{\infty} \left(\frac{\lambda}{c\mu} \right)^{j-c}.$$

Letting $k = j - c$, we obtain

$$P_{\text{wait}} = p_0 \cdot \frac{1}{c!} \left(\frac{\lambda}{\mu} \right)^c \sum_{k=0}^{\infty} \left(\frac{\lambda}{c\mu} \right)^k.$$

This is a geometric series, giving

$$P_{\text{wait}} = p_0 \cdot \frac{1}{c!} \left(\frac{\lambda}{\mu} \right)^c \cdot \frac{1}{1 - \frac{\lambda}{c\mu}}.$$

Substituting the expression for p_0 from normalization yields the Erlang-C formula.

Expected Queue Length

The expected number of customers in the queue is

$$L_q = \sum_{j=c}^{\infty} (j - c) p_j.$$

Using the expression for p_j , we obtain

$$L_q = p_0 \cdot \frac{1}{c!} \left(\frac{\lambda}{\mu} \right)^c \sum_{j=c}^{\infty} (j - c) \left(\frac{\lambda}{c\mu} \right)^{j-c}.$$

Let $k = j - c$, giving

$$L_q = p_0 \cdot \frac{1}{c!} \left(\frac{\lambda}{\mu} \right)^c \sum_{k=0}^{\infty} k \left(\frac{\lambda}{c\mu} \right)^k.$$

Using the identity

$$\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2}, \quad |x| < 1,$$

we obtain

$$L_q = p_0 \cdot \frac{1}{c!} \left(\frac{\lambda}{\mu} \right)^c \cdot \frac{\frac{\lambda}{c\mu}}{\left(1 - \frac{\lambda}{c\mu} \right)^2}.$$

Recognizing that

$$P_{\text{wait}} = p_0 \cdot \frac{1}{c!} \left(\frac{\lambda}{\mu} \right)^c \cdot \frac{1}{1 - \frac{\lambda}{c\mu}},$$

we simplify to

$$L_q = \frac{P_{\text{wait}} \cdot \frac{\lambda}{c\mu}}{1 - \frac{\lambda}{c\mu}}.$$

Thus,

$$L_q = \frac{P_{\text{wait}} \cdot \rho}{1 - \rho}.$$

Final Expression

Finally, the expected number of customers in the system is

$$\mathbb{E}[N] = L = L_q + L_s = \frac{\lambda}{\mu} + \frac{P_{\text{wait}} \cdot \rho}{1 - \rho}.$$

3.8 Simulation Approach

To validate the analytical results derived for the $M/M/c$ queue, we simulate the system as a continuous-time Markov process.

At any state n :

- Arrivals occur at rate λ
- Departures occur at rate $\min(n, c)\mu$

Thus, the total transition rate is

$$\lambda + \min(n, c)\mu.$$

The simulation proceeds by:

- Sampling the time to the next event using an exponential distribution
- Determining whether the event is an arrival or a departure
- Updating the system state accordingly

3.9 Core Simulation

The system is simulated using the following structure:

```
arrival_rate <- lambda
service_rate <- min(n, c) * mu
total_rate <- arrival_rate + service_rate

dt <- rexp(1, total_rate)
```

The event type is determined probabilistically:

```
if(runif(1) < arrival_rate / total_rate){
  n <- n + 1
} else {
  n <- max(0, n - 1)
}
```

3.10 Estimation of Performance Measures

We estimate quantities using time averages. Let $\{t_i\}$ denote event times and $\{X_i\}$ the corresponding states.

Expected Number in System

$$L = \frac{\sum_i X_i \Delta t_i}{\sum_i \Delta t_i}$$

```
L_hat <- sum(states * durations) / total_time
```

Expected Queue Length

Only customers exceeding server capacity contribute to the queue:

$$L_q = \frac{\sum_i \max(X_i - c, 0) \Delta t_i}{\sum_i \Delta t_i}$$

```
queue <- pmax(states - c, 0)
Lq_hat <- sum(queue * durations) / total_time
```

Probability of Waiting

$$P_{\text{wait}} = \mathbb{P}(X_t \geq c)$$

```
P_wait_hat <- sum(durations[states >= c]) / total_time
```

3.11 Verification of Stationary Distribution

The stationary probabilities are estimated as:

$$\hat{p}_j = \frac{\text{time spent in state } j}{\text{total time}}.$$

```
p_hat <- sapply(0:max_state, function(j)
  sum(durations[states == j]) / total_time)
```

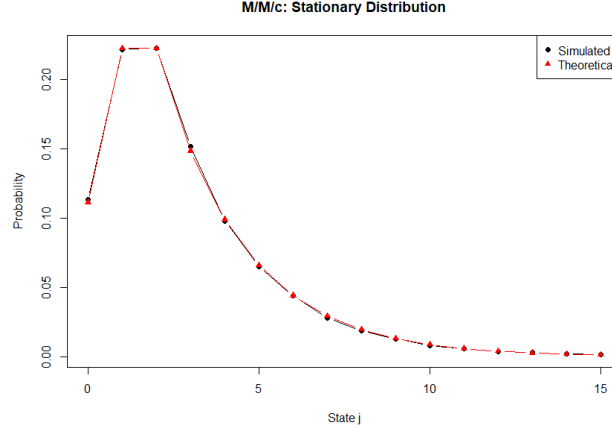
These are compared with the theoretical values derived earlier.

3.12 Results

For $\lambda = 4$, $\mu = 2$, and $c = 3$, we obtain the following:

Metric	Simulated	Theoretical
$L = \mathbb{E}[N]$	2.8733	2.8889
L_q	0.8778	0.8889
P_{wait}	0.4430	0.4444

A graphical comparison of $\hat{p}_j$ and p_j further confirms the match between simulation and theory.



3.13 Conclusion

The simulation validates the analytical expressions derived for the $M/M/c$ queue. The empirical estimates of all key performance measures closely match their theoretical counterparts.

This demonstrates convergence of the system to its stationary distribution and confirms the correctness of the balance-based derivations.

Furthermore, the results illustrate how increasing the number of servers reduces congestion, while higher arrival rates lead to significant increases in queue length and waiting probability.

Finally, this simulation framework provides a foundation for analyzing more complex systems such as $M/G/c$ queues, where closed-form solutions are not available.

4 M/G/1 Queue: Analysis

4.1 Model Description

We now consider an $M/G/1$ queueing system with a single server.

- Arrivals follow a Poisson process with rate λ
- Service times are independent and identically distributed with a general distribution G
- There is one server
- Customers are served in first-come-first-served order

Let $\{X_t\}_{t \geq 0}$ denote the number of customers in the system at time t .

Let S denote a generic service time. We assume

$$\mathbb{E}[S] < \infty$$

and, for the waiting-time formulas,

$$\mathbb{E}[S^2] < \infty.$$

4.2 Arrival and Service Structure

Since arrivals follow a Poisson process with rate λ , for small $h > 0$,

$$\mathbb{P}(\text{one arrival in } (t, t+h]) = \lambda h + o(h),$$

and

$$\mathbb{P}(\text{two or more arrivals in } (t, t+h]) = o(h).$$

The interarrival times are independent exponential random variables with rate λ .

If A denotes a generic interarrival time, then

$$A \sim \text{Exp}(\lambda),$$

and hence

$$\mathbb{E}[A] = \frac{1}{\lambda}.$$

Unlike the $M/M/1$ queue, the service times are not necessarily exponential. Therefore, the process $\{X_t\}_{t \geq 0}$ alone is generally not a continuous-time Markov chain.

This is because, when the server is busy, the future evolution depends not only on the number of customers in the system, but also on the remaining service time of the customer currently in service.

4.3 Why the Queue-Length Process is Not Markov

In an $M/M/1$ queue, service times are exponential. The exponential distribution has the memoryless property, so the remaining service time has the same distribution regardless of how long the current service has already lasted.

However, in an $M/G/1$ queue, service times have a general distribution. Therefore, the remaining service time depends on the age of the service already completed.

Thus, knowing only X_t is not enough to describe the future evolution of the system.

Hence, in general,

$$\{X_t\}_{t \geq 0}$$

is not a Markov chain.

To analyze the system, we instead study embedded processes such as:

- the queue length immediately after departures;
- the waiting time recursion seen by successive customers;
- the workload process.

4.4 Traffic Intensity and Stability Condition

The traffic intensity is defined as

$$\rho = \lambda \mathbb{E}[S].$$

This quantity represents the long-run fraction of time the server is busy.

For the queue to be stable, the average amount of work arriving per unit time must be less than the service capacity of the server.

Since work arrives at rate

$$\lambda \mathbb{E}[S],$$

and the server can process one unit of work per unit time, the stability condition is

$$\rho < 1.$$

Thus,

$$\lambda \mathbb{E}[S] < 1.$$

If $\rho \geq 1$, then work arrives at least as fast as it can be processed, and the queue does not have a steady-state distribution.

4.5 Embedded Queue-Length Process at Departures

Let Q_n denote the number of customers left behind immediately after the n -th departure.

During the service time of the next customer, new arrivals occur.

Let A_{n+1} denote the number of arrivals during the service time of the next customer.

If $Q_n \geq 1$, then one of the waiting customers enters service immediately after the n -th departure. After this service is completed, the queue length decreases by one due to the departure and increases by A_{n+1} due to arrivals during service.

Thus,

$$Q_{n+1} = Q_n - 1 + A_{n+1}, \quad Q_n \geq 1.$$

If $Q_n = 0$, then the system becomes empty after the n -th departure. The next customer arrives after an idle period and begins service immediately. At the next departure, the number left behind is simply the number of arrivals during that service time.

Thus,

$$Q_{n+1} = A_{n+1}, \quad Q_n = 0.$$

Combining both cases,

$$Q_{n+1} = (Q_n - 1)^+ + A_{n+1},$$

where

$$x^+ = \max(x, 0).$$

4.6 Distribution of Arrivals During a Service Time

Conditional on a service time $S = s$, the number of arrivals during that service time follows a Poisson distribution with mean λs .

Therefore,

$$A \mid S = s \sim \text{Poisson}(\lambda s).$$

Hence,

$$\mathbb{P}(A = k \mid S = s) = e^{-\lambda s} \frac{(\lambda s)^k}{k!}.$$

Unconditionally,

$$\mathbb{P}(A = k) = \int_0^\infty e^{-\lambda s} \frac{(\lambda s)^k}{k!} dG(s).$$

Thus, the embedded queue-length process $\{Q_n\}_{n \geq 0}$ is a discrete-time Markov chain.

4.7 Mean Number of Arrivals During Service

Using conditional expectation,

$$\mathbb{E}[A] = \mathbb{E}[\mathbb{E}[A \mid S]].$$

Since

$$A \mid S = s \sim \text{Poisson}(\lambda s),$$

we have

$$\mathbb{E}[A \mid S = s] = \lambda s.$$

Therefore,

$$\mathbb{E}[A] = \mathbb{E}[\lambda S] = \lambda \mathbb{E}[S] = \rho.$$

Thus, the mean number of arrivals during a service time is exactly the traffic intensity.

4.8 Workload and Lindley's Recursion

Let W_n denote the waiting time in queue of the n -th customer.

Let S_n be the service time of the n -th customer.

Let T_n be the interarrival time between customer n and customer $n + 1$.

Suppose customer n waits W_n before service. Then the total amount of work ahead of customer $n + 1$ just before their arrival is

$$W_n + S_n - T_n.$$

If this quantity is positive, customer $n + 1$ has to wait. If it is negative, the server has become idle before customer $n + 1$ arrives, so the waiting time is zero.

Therefore,

$$W_{n+1} = \max\{0, W_n + S_n - T_n\}.$$

Equivalently,

$$W_{n+1} = (W_n + S_n - T_n)^+.$$

This is called Lindley's recursion.

4.9 Pollaczek–Khinchine Formula

For a stable $M/G/1$ queue, the expected waiting time in queue is given by the Pollaczek–Khinchine formula:

$$\mathbb{E}[W_q] = \frac{\lambda \mathbb{E}[S^2]}{2(1 - \rho)}.$$

Here,

$$\rho = \lambda \mathbb{E}[S].$$

The total time spent in the system is

$$W = W_q + S.$$

Taking expectations,

$$\mathbb{E}[W] = \mathbb{E}[W_q] + \mathbb{E}[S].$$

Therefore,

$$\mathbb{E}[W] = \mathbb{E}[S] + \frac{\lambda \mathbb{E}[S^2]}{2(1 - \rho)}.$$

4.10 Expected Number of Customers

Let

$$L = \mathbb{E}[N]$$

denote the expected number of customers in the system.

Let

$$L_q$$

denote the expected number of customers waiting in queue.

By Little's Law,

$$L = \lambda \mathbb{E}[W],$$

and

$$L_q = \lambda \mathbb{E}[W_q].$$

Using the Pollaczek–Khinchine formula,

$$L_q = \lambda \cdot \frac{\lambda \mathbb{E}[S^2]}{2(1 - \rho)}.$$

Thus,

$$L_q = \frac{\lambda^2 \mathbb{E}[S^2]}{2(1 - \rho)}.$$

Also,

$$L = \lambda \left(\mathbb{E}[S] + \frac{\lambda \mathbb{E}[S^2]}{2(1 - \rho)} \right).$$

Since

$$\rho = \lambda \mathbb{E}[S],$$

we obtain

$$L = \rho + \frac{\lambda^2 \mathbb{E}[S^2]}{2(1 - \rho)}.$$

Thus,

$$L = \rho + L_q.$$

4.11 Role of Service-Time Variability

The formula

$$\mathbb{E}[W_q] = \frac{\lambda \mathbb{E}[S^2]}{2(1 - \rho)}$$

shows that the expected waiting time depends on the second moment of the service time.

Since

$$\mathbb{E}[S^2] = \text{Var}(S) + (\mathbb{E}[S])^2,$$

we can write

$$\mathbb{E}[W_q] = \frac{\lambda (\text{Var}(S) + (\mathbb{E}[S])^2)}{2(1 - \rho)}.$$

Therefore, even if two systems have the same arrival rate and the same mean service time, the system with larger service-time variance will have larger expected waiting time.

4.12 Special Case: M/M/1 Queue

For an $M/M/1$ queue, service times are exponential with rate μ .

Thus,

$$S \sim \text{Exp}(\mu).$$

Hence,

$$\mathbb{E}[S] = \frac{1}{\mu},$$

and

$$\mathbb{E}[S^2] = \frac{2}{\mu^2}.$$

Also,

$$\rho = \lambda \mathbb{E}[S] = \frac{\lambda}{\mu}.$$

Substituting into the Pollaczek–Khinchine formula,

$$\mathbb{E}[W_q] = \frac{\lambda \frac{2}{\mu^2}}{2(1 - \lambda/\mu)}.$$

Therefore,

$$\mathbb{E}[W_q] = \frac{\lambda}{\mu^2(1 - \lambda/\mu)}.$$

Since

$$1 - \frac{\lambda}{\mu} = \frac{\mu - \lambda}{\mu},$$

we get

$$\mathbb{E}[W_q] = \frac{\lambda}{\mu(\mu - \lambda)}.$$

The expected total time in system is

$$\mathbb{E}[W] = \mathbb{E}[W_q] + \frac{1}{\mu}.$$

Thus,

$$\mathbb{E}[W] = \frac{1}{\mu - \lambda}.$$

4.13 Special Case: M/D/1 Queue

For an $M/D/1$ queue, service times are deterministic.

Suppose

$$S = s$$

with probability 1.

Then

$$\mathbb{E}[S] = s,$$

and

$$\mathbb{E}[S^2] = s^2.$$

The traffic intensity is

$$\rho = \lambda s.$$

Using the Pollaczek–Khinchine formula,

$$\mathbb{E}[W_q] = \frac{\lambda s^2}{2(1 - \lambda s)}.$$

The expected total time in system is

$$\mathbb{E}[W] = s + \frac{\lambda s^2}{2(1 - \lambda s)}.$$

4.14 Example

Suppose orders arrive at a trading engine according to a Poisson process with rate

$$\lambda = 4.$$

Suppose service times are generally distributed with

$$\mathbb{E}[S] = 0.2,$$

and

$$\text{Var}(S) = 0.04.$$

Then

$$\rho = \lambda \mathbb{E}[S] = 4(0.2) = 0.8.$$

Since

$$\rho < 1,$$

the system is stable.

Now,

$$\mathbb{E}[S^2] = \text{Var}(S) + (\mathbb{E}[S])^2.$$

Therefore,

$$\mathbb{E}[S^2] = 0.04 + (0.2)^2 = 0.08.$$

Using Pollaczek–Khinchine,

$$\mathbb{E}[W_q] = \frac{\lambda \mathbb{E}[S^2]}{2(1 - \rho)}.$$

Thus,

$$\mathbb{E}[W_q] = \frac{4(0.08)}{2(1 - 0.8)} = \frac{0.32}{0.4} = 0.8.$$

The expected total time in the system is

$$\mathbb{E}[W] = 0.8 + 0.2 = 1.0.$$

Also,

$$L_q = \lambda \mathbb{E}[W_q] = 4(0.8) = 3.2.$$

Finally,

$$L = \lambda \mathbb{E}[W] = 4(1.0) = 4.$$

4.15 Simulation Approach

To validate the analytical results derived for the $M/G/1$ queue, we simulate the system using Lindley’s recursion.

The simulation proceeds by:

- sampling interarrival times from an exponential distribution with rate λ ;
- sampling service times from a chosen general distribution;
- recursively computing waiting times using Lindley’s recursion;
- estimating average waiting time, system time, queue length, and server utilization.

4.16 Core Simulation

The waiting-time recursion is

$$W_{n+1} = \max\{0, W_n + S_n - T_n\}.$$

A basic implementation in R is:

```
lambda <- 4
n <- 100000

# Example: gamma service times
shape <- 1
scale <- 0.2

interarrival <- rexp(n, rate = lambda)
service <- rgamma(n, shape = shape, scale = scale)

waiting <- numeric(n)

for(i in 1:(n-1)){
  waiting[i+1] <- max(0, waiting[i] + service[i] -
    interarrival[i])
}
```

4.17 Estimation of Performance Measures

The empirical expected waiting time in queue is estimated by

$$\widehat{W}_q = \frac{1}{n} \sum_{i=1}^n W_i.$$

```
Wq_hat <- mean(waiting)
```

The empirical expected total time in system is estimated by

$$\widehat{W} = \frac{1}{n} \sum_{i=1}^n (W_i + S_i).$$

```
W_hat <- mean(waiting + service)
```

Using Little's Law, we estimate

$$\widehat{L}_q = \lambda \widehat{W}_q$$

and

$$\widehat{L} = \lambda \widehat{W}.$$

```
Lq_hat <- lambda * Wq_hat
L_hat <- lambda * W_hat
```

The theoretical values are computed as:

```
mean_S <- mean(service)
second_moment_S <- mean(service^2)
rho <- lambda * mean_S

Wq_theory <- lambda * second_moment_S / (2 * (1 - rho))
W_theory <- Wq_theory + mean_S

Lq_theory <- lambda * Wq_theory
L_theory <- lambda * W_theory
```

4.18 Results

For $\lambda = 4$ and gamma service times with shape 1 and scale 0.2, we have approximately

$$\mathbb{E}[S] = 0.2,$$

and

$$\mathbb{E}[S^2] = 0.08.$$

The theoretical values are:

$$\rho = 0.8,$$

$$\mathbb{E}[W_q] = 0.8,$$

$$\mathbb{E}[W] = 1.0,$$

$$L_q = 3.2,$$

and

$$L = 4.0.$$

A typical simulation gives values close to these theoretical quantities.

Metric	Simulated	Theoretical
$\mathbb{E}[W_q]$	close to 0.8	0.8
$\mathbb{E}[W]$	close to 1.0	1.0
L_q	close to 3.2	3.2
L	close to 4.0	4.0

4.19 Conclusion

The $M/G/1$ queue generalizes the $M/M/1$ queue by allowing service times to follow an arbitrary distribution.

Unlike the $M/M/1$ queue, the queue-length process $\{X_t\}_{t \geq 0}$ is generally not Markov because the remaining service time matters.

However, the system can still be analyzed using embedded processes and workload recursions.

The key stability condition is

$$\rho = \lambda \mathbb{E}[S] < 1.$$

The central performance formula is the Pollaczek–Khinchine formula:

$$\mathbb{E}[W_q] = \frac{\lambda \mathbb{E}[S^2]}{2(1 - \rho)}.$$

This formula shows that congestion depends not only on the mean service time, but also on the variability of service times.

The simulation based on Lindley’s recursion validates the theoretical expressions and provides a practical way to study systems with general service-time distributions.

5 M/G/c Queue: Simulation-Based Analysis

5.1 Model Description

We now consider an $M/G/c$ queueing system with c identical parallel servers.

- Arrivals follow a Poisson process with rate λ
- Service times are independent and identically distributed with a general distribution G
- There are c identical servers
- Customers are served in first-come-first-served order

Let $\{X_t\}_{t \geq 0}$ denote the number of customers in the system at time t .

Unlike the $M/M/c$ queue, service times are not necessarily exponential. Therefore, the number of customers in the system alone is generally not enough to describe the future evolution of the system.

This is because the future depends not only on how many customers are present, but also on the remaining service times of customers currently being served.

Hence, for an $M/G/c$ queue, we use a simulation-based approach.

5.2 Why Simulation is Needed

For an $M/M/c$ queue, service times are exponential. The memoryless property allows the queue length process to be treated as a continuous-time Markov chain.

However, in an $M/G/c$ queue, service times follow a general distribution. This means that the remaining service time of a customer depends on how long that customer has already been in service.

Therefore, the process $\{X_t\}_{t \geq 0}$ is generally not Markov.

Closed-form expressions for quantities such as expected queue length, waiting time, and probability of waiting are usually not available in simple form.

Simulation allows us to estimate these performance measures directly.

5.3 Quantities to Estimate

The main quantities of interest are:

- L : expected number of customers in the system
- L_q : expected number of customers waiting in queue
- W : expected time spent in the system
- W_q : expected waiting time in queue
- P_{wait} : probability that an arriving customer has to wait
- server utilization
- distribution of waiting times
- distribution of queue lengths

These quantities can be estimated from a long simulation run.

5.4 Stability Condition

Let S denote a generic service time.

The traffic intensity is

$$\rho = \frac{\lambda \mathbb{E}[S]}{c}.$$

For the system to be stable, the average amount of work arriving per unit time must be less than the total service capacity of the c servers.

Thus, the stability condition is

$$\rho < 1.$$

Equivalently,

$$\lambda \mathbb{E}[S] < c.$$

If $\rho \geq 1$, then work arrives at least as fast as the servers can process it, and the queue tends to grow without bound.

5.5 Event-Based Simulation Idea

The simulation is based on tracking events over time.

There are two types of events:

- arrival events;
- departure events.

At each step, the simulation moves to the next event time.

For each arriving customer:

- if at least one server is free, the customer immediately enters service;
- if all servers are busy, the customer joins the queue;
- the customer's waiting time is recorded;
- the customer's departure time is determined once service begins.

For each departure:

- the server becomes free;
- if the queue is nonempty, the next waiting customer immediately begins service;
- a new departure time is scheduled for that server.

5.6 Alternative Customer-Based Simulation

A simpler way to simulate an $M/G/c$ queue is to process customers one by one.

Let

$$A_i$$

be the arrival time of customer i .

Let

$$S_i$$

be the service time of customer i .

Let

$$B_i$$

be the service start time of customer i .

Let

$$D_i$$

be the departure time of customer i .

For each customer, we assign the customer to the server that becomes available the earliest.

Let

$$F_k$$

denote the next free time of server k , for

$$k = 1, 2, \dots, c.$$

When customer i arrives, we find

$$k^* = \arg \min_{1 \leq k \leq c} F_k.$$

The service start time is

$$B_i = \max(A_i, F_{k^*}).$$

The waiting time in queue is

$$W_{q,i} = B_i - A_i.$$

The departure time is

$$D_i = B_i + S_i.$$

Then we update

$$F_{k^*} = D_i.$$

This gives a direct simulation algorithm for the $M/G/c$ queue.

5.7 Core Simulation Algorithm

The algorithm is:

1. Generate interarrival times:

$$T_i \sim \text{Exp}(\lambda).$$

2. Construct arrival times:

$$A_i = \sum_{r=1}^i T_r.$$

3. Generate service times:

$$S_i \sim G.$$

4. Initialize server free times:

$$F_1 = F_2 = \dots = F_c = 0.$$

5. For each customer i :

- (a) find the server with the smallest free time;
- (b) start service at the later of arrival time and server free time;
- (c) compute waiting time;
- (d) compute departure time;
- (e) update that server's free time.

5.8 Core Simulation Code

A basic implementation in R is given below.

```
set.seed(42)

lambda <- 4
c <- 3
n <- 100000

# Example: gamma service times
shape <- 2
scale <- 0.3

# Generate interarrival times and arrival times
interarrival <- rexp(n, rate = lambda)
arrival <- cumsum(interarrival)

# Generate service times
service <- rgamma(n, shape = shape, scale = scale)

# Server free times
server_free <- rep(0, c)

# Store results
start_service <- numeric(n)
departure <- numeric(n)
waiting <- numeric(n)
server_assigned <- numeric(n)

for(i in 1:n){

  # Find earliest available server
  k <- which.min(server_free)

  # Service starts when both customer has arrived and server
  # is free
  start_service[i] <- max(arrival[i], server_free[k])

  # Waiting time
  waiting[i] <- start_service[i] - arrival[i]

  # Departure time
  departure[i] <- start_service[i] + service[i]

  # Update server free time
  server_free[k] <- departure[i]

  # Store assigned server
  server_assigned[i] <- k
```

```
}
```

5.9 Estimating Waiting-Time Quantities

The empirical expected waiting time in queue is

$$\widehat{W}_q = \frac{1}{n} \sum_{i=1}^n W_{q,i}.$$

```
Wq_hat <- mean(waiting)
```

The empirical expected total time in system is

$$\widehat{W} = \frac{1}{n} \sum_{i=1}^n (D_i - A_i).$$

```
system_time <- departure - arrival
W_hat <- mean(system_time)
```

The probability of waiting is estimated by

$$\widehat{P}_{\text{wait}} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}_{\{W_{q,i} > 0\}}.$$

```
P_wait_hat <- mean(waiting > 0)
```

5.10 Estimating Queue Length and System Size

To estimate L and L_q , we use time averages.

Let event times be all arrival and departure times sorted in increasing order.

Between two consecutive event times, the number of customers in the system remains constant.

Thus,

$$\widehat{L} = \frac{\sum_i X_i \Delta t_i}{\sum_i \Delta t_i}.$$

Similarly,

$$\widehat{L}_q = \frac{\sum_i Q_i \Delta t_i}{\sum_i \Delta t_i},$$

where

$$Q_i = \max(X_i - c, 0).$$

```

events <- data.frame(
  time = c(arrival, departure),
  type = c(rep(1, n), rep(-1, n))
)

events <- events[order(events$time), ]

current_n <- 0
last_time <- 0

area_system <- 0
area_queue <- 0

for(i in 1:nrow(events)){

  time <- events$time[i]
  dt <- time - last_time

  area_system <- area_system + current_n * dt
  area_queue <- area_queue + max(current_n - c, 0) * dt

  current_n <- current_n + events$type[i]
  last_time <- time
}

total_time <- max(departure)

L_hat <- area_system / total_time
Lq_hat <- area_queue / total_time

```

5.11 Estimating Server Utilization

The utilization of the system can be estimated by measuring the fraction of total server capacity used.

The total busy time is

$$\sum_{i=1}^n S_i.$$

The total available server time is

$$cT,$$

where T is the total simulation time.

Thus,

$$\hat{\rho} = \frac{\sum_{i=1}^n S_i}{cT}.$$

```
rho_hat <- sum(service) / (c * total_time)
```

The theoretical traffic intensity is

$$\rho = \frac{\lambda \mathbb{E}[S]}{c}.$$

```
rho_theory <- lambda * mean(service) / c
```

5.12 Checking Little's Law

The simulation estimates should approximately satisfy Little's Law:

$$L \approx \lambda W,$$

and

$$L_q \approx \lambda W_q.$$

In code:

```
lambda_eff <- n / total_time  
L_little <- lambda_eff * W_hat  
Lq_little <- lambda_eff * Wq_hat
```

Here λ_{eff} is the empirical arrival rate observed in the simulation.

5.13 Results

For example, suppose

$$\lambda = 4,$$

$$c = 3,$$

and service times follow a gamma distribution with

$$\text{shape} = 2, \quad \text{scale} = 0.3.$$

Then

$$\mathbb{E}[S] = 2(0.3) = 0.6.$$

The traffic intensity is

$$\rho = \frac{\lambda \mathbb{E}[S]}{c} = \frac{4(0.6)}{3} = 0.8.$$

Since

$$\rho < 1,$$

the system is stable.

A typical simulation gives estimates of the form:

Metric	Simulated Estimate
$\widehat{W}_q$	estimated from waiting times
$\widehat{W}$	estimated from system times
$\widehat{L}_q$	estimated by time average
$\widehat{L}$	estimated by time average
$\widehat{P}_{\text{wait}}$	estimated by fraction of customers who wait
$\widehat{\rho}$	estimated by busy time fraction

Unlike the $M/M/c$ queue, there is no simple closed-form expression included here for comparison. Instead, the simulation itself is used to study the behaviour of the system.

5.14 Complete R Code

```
set.seed(42)

# Parameters
lambda <- 4
c <- 3
n <- 100000

# Service-time distribution: Gamma(shape, scale)
shape <- 2
scale <- 0.3

# Generate arrivals
interarrival <- rexp(n, rate = lambda)
arrival <- cumsum(interarrival)

# Generate service times
service <- rgamma(n, shape = shape, scale = scale)

# Check stability
mean_service <- mean(service)
rho_theory <- lambda * mean_service / c

cat("Estimated mean service time:", mean_service, "\n")
cat("Estimated traffic intensity:", rho_theory, "\n")

if(rho_theory >= 1){
  warning("System may be unstable: rho >= 1")
}

# Initialize servers
server_free <- rep(0, c)

# Storage
start_service <- numeric(n)
```



```

departure <- numeric(n)
waiting <- numeric(n)
server_assigned <- numeric(n)

# Customer-based simulation
for(i in 1:n){

  k <- which.min(server_free)

  start_service[i] <- max(arrival[i], server_free[k])
  waiting[i] <- start_service[i] - arrival[i]
  departure[i] <- start_service[i] + service[i]

  server_free[k] <- departure[i]
  server_assigned[i] <- k
}

# Customer-average quantities
system_time <- departure - arrival

Wq_hat <- mean(waiting)
W_hat <- mean(system_time)
P_wait_hat <- mean(waiting > 0)

# Time-average quantities
events <- data.frame(
  time = c(arrival, departure),
  type = c(rep(1, n), rep(-1, n))
)

events <- events[order(events$time), ]

current_n <- 0
last_time <- 0
area_system <- 0
area_queue <- 0

for(i in 1:nrow(events)){

  time <- events$time[i]
  dt <- time - last_time

  area_system <- area_system + current_n * dt
  area_queue <- area_queue + max(current_n - c, 0) * dt

  current_n <- current_n + events$type[i]
  last_time <- time
}

total_time <- max(departure)

```

```

L_hat <- area_system / total_time
Lq_hat <- area_queue / total_time

# Utilization
rho_hat <- sum(service) / (c * total_time)

# Effective arrival rate
lambda_eff <- n / total_time

# Little's Law checks
L_little <- lambda_eff * W_hat
Lq_little <- lambda_eff * Wq_hat

# Print results
cat("\nSimulation Results\n")
cat("-----\n")
cat("Wq_hat:", Wq_hat, "\n")
cat("W_hat:", W_hat, "\n")
cat("Lq_hat:", Lq_hat, "\n")
cat("L_hat:", L_hat, "\n")
cat("P_wait_hat:", P_wait_hat, "\n")
cat("rho_hat:", rho_hat, "\n")

cat("\nLittle's Law Checks\n")
cat("-----\n")
cat("L_hat:", L_hat, "| lambda_eff * W_hat:", L_little, "\n")
cat("Lq_hat:", Lq_hat, "| lambda_eff * Wq_hat:", Lq_little, "\n")

# Plots

# Waiting time histogram
hist(waiting,
      breaks = 50,
      probability = TRUE,
      main = "Waiting Time Distribution in M/G/c Queue",
      xlab = "Waiting time")

# System time histogram
hist(system_time,
      breaks = 50,
      probability = TRUE,
      main = "System Time Distribution in M/G/c Queue",
      xlab = "System time")

# Server assignment counts
barplot(table(server_assigned),
        main = "Number of Customers Served by Each Server",

```

```
xlab = "Server",
ylab = "Number_of_customers")
```

5.15 Interpretation

The simulation estimates the main performance measures of the $M/G/c$ queue directly from sample paths.

The customer-level quantities are estimated using customer averages:

$$\widehat{W}_q = \frac{1}{n} \sum_{i=1}^n W_{q,i},$$

and

$$\widehat{W} = \frac{1}{n} \sum_{i=1}^n (D_i - A_i).$$

The time-average quantities are estimated by integrating the sample path:

$$\widehat{L} = \frac{1}{T} \int_0^T X_t dt,$$

and

$$\widehat{L}_q = \frac{1}{T} \int_0^T (X_t - c)^+ dt.$$

The results should satisfy Little's Law approximately:

$$\widehat{L} \approx \lambda_{\text{eff}} \widehat{W},$$

and

$$\widehat{L}_q \approx \lambda_{\text{eff}} \widehat{W}_q.$$

If the simulation is long enough and the system is stable, these approximations should be close.

5.16 Conclusion

The $M/G/c$ queue is difficult to analyze using simple closed-form formulas because service times are generally distributed and multiple servers are present.

Therefore, simulation is a natural and powerful approach.

The customer-based simulation assigns each arriving customer to the earliest available server and records waiting times, service start times, and departure times.

From these simulated paths, we estimate waiting times, system times, queue lengths, probability of waiting, and utilization.

This simulation framework can be used for many practical systems, including:

- airport check-in counters;
- bank service desks;
- order execution engines;
- transaction processing systems;
- compute clusters;
- call centers;
- risk calculation servers.

Thus, even when closed-form theory is unavailable, simulation allows us to study congestion, delay, and server utilization in realistic multi-server systems.